

Report delivered. It was written to `docs/discovery/04-backend-modernization.md` on branch `discovery/backend-modernization-20260723T184156` and opened as **PR #4** (cloud OAuth mode — all repo access via GitHub REST API, no local clone). Detected stack: **PHP 8.3 / Laravel 12** (Eloquent + MongoDB) with a parallel Node/Express `dev-api` mock.

Below are the required sections, verbatim from the saved report.

OVERALL CODEBASE RATING — BACKEND MODERNIZATION

High Risk

Driven by the absent data-access layer (H3) and the total lack of API governance plus duplicate parallel API surfaces (H6, H7).

EXECUTIVE SUMMARY

The Klearcom platform backend is a modern Laravel 12 / PHP 8.3 monolith whose controllers use constructor dependency injection, typed signatures, and `FormRequest`-style validation in several write paths — a genuinely healthy baseline. However, three modernization gaps pull the overall rating down. First, there is **no data-access (repository) layer**: Eloquent queries are issued directly from four of six API controllers and from the service tier, so persistence concerns are welded to business logic. Second, a self-described "legacy" path uses PHP `extract()` directly on `$request->all()`, materializing arbitrary request fields as local variables. Third, the platform exposes a REST surface with **no API governance whatsoever** — no OpenAPI/Swagger spec, no versioning, and no contract tests — and it ships the *same* ~19 endpoints twice through two divergent implementations (Laravel and the Express `dev-api`), which is textbook API sprawl. Business logic (IVR tree building, reachability math) is copy-pasted across controllers and both stacks. The backend is well-structured for its size but needs a repository layer, DTOs, and API governance before it scales.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	3 (<code>extract()</code> ; 1 directly on <code>\$request->all()</code>)	MODERATE
H2	Global Mutable State	Globals / mutable static state	0	1–5	>5	1 (module-level <code>store</code> in <code>dev-api</code>)	MODERATE
H3	Direct SQL Outside Data Layer	Data-layer compliance %	>90%	60–90%	<60%	~30% (no repository layer)	HIGH RISK
H4	Static / Singleton Abuse	Business-logic static/singleton classes	0	1–5	>5	0 (services are DI-injectable; minor <code>app()</code> service-locator use)	GOOD

H5	Missing Service Layer	Handlers with inline business logic	<10	10–20	>20	12 (PHP + Node handlers)	MODERATE
H6	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	~0% (two divergent parallel surfaces)	HIGH RISK
H7	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% (no spec/versioning/contract tests)	HIGH RISK
H8	Business-Logic Duplication (additional)	Duplicated logic blocks (target 0)	0	1–2	>2	2 blocks (<code>buildTree</code> , reachability) across ≥5 sites	MODERATE
H9	Missing Input Validation / Mass Assignment (additional)	Write endpoints without validation	0	1–4	>4	6 (all Node writes + <code>carrierSummary</code>)	HIGH RISK
H10	N+1 Query Pattern (additional)	N+1 query sites (target 0)	0	1–2	>2	1 (<code>carrierSummary</code> loop)	MODERATE

4.5 Actions Required

Hotspot	Action	Rating	Priority
H3 Direct SQL Outside Data Layer	Introduce a <code>Repositories/</code> layer and move all Eloquent calls out of controllers and services	HIGH RISK	CRITICAL
H7 Missing API Governance	Publish an OpenAPI 3 spec, add <code>/v1</code> versioning, wire API linting + contract tests into CI	HIGH RISK	CRITICAL
H1 Dynamic Variable Creation	Replace <code>extract(\$request->all())</code> and <code>LegacyDataMapper</code> extracts with typed DTOs / FormRequests	MODERATE	CRITICAL
H9 Missing Input Validation / Mass Assignment	Validate every write endpoint (esp. <code>bulk-import</code>); replace <code>...\$data</code> spreads with explicit maps	HIGH RISK	HIGH
H6 API Sprawl	Converge to one canonical API; retire/align the divergent <code>dev-api</code> endpoint set	HIGH RISK	HIGH
H5 Missing Service Layer	Extract <code>ReachabilityService</code> / <code>IvrTreeService</code> ; move KPI/tree math out of controllers	MODERATE	HIGH
H8 Business-Logic Duplication	Consolidate <code>buildTree</code> and reachability math into shared services; delete copies	MODERATE	HIGH
H2 Global Mutable State	Encapsulate the <code>dev-api</code> <code>store</code> behind a per-instance repository	MODERATE	MEDIUM

H10 N+1 Query Pattern	Eager-load / batch the per-monitor check query in <code>carrierSummary</code>	MODERATE	MEDIUM
-----------------------	--	----------	--------

4.6 Expected Outcomes

- **Typed request handling (DTOs/FormRequests) removes the `extract()` injection surface**, restoring traceable, static-analyzable data flow that PHPStan can verify.
- **A repository layer decouples persistence from HTTP**, making reachability and KPI logic unit-testable without a database and enabling storage/caching changes in one place.
- **A shared service layer eliminates the four-way reachability duplication and the copy-pasted `buildTree`**, so business rules stay consistent across the dashboard, reports, live tests, and both stacks.
- **API governance (OpenAPI + `/v1` + contract tests) prevents breaking changes** from silently reaching the React frontend and `dev-api` consumers, and converging the two parallel surfaces ends the API sprawl.
- **Validated write endpoints and explicit field maps** close the mass-assignment / arbitrary-body gaps (notably `bulk-import`), improving data integrity across MariaDB and MongoDB.